RENTORA

Real-Time Property Rental, Amenity & Maintenance Management Platform

PROJECT INTERNSHIP REPORT

Submitted in partial fulfillment of the requirements

for the Unified Mentor Internship Program

Submitted By

Rishabh Singh

Under the Guidance of

Unified Mentor Technical Panel

Unified Mentor

Internship Program

August 2, 2026

Acknowledgement

I express my sincere gratitude to the technical mentors and evaluation panel of the **Unified Mentor Internship Program** for their constant guidance, valuable feedback, and technical support throughout the architectural design, implementation, and benchmarking phases of the Rentora platform.

Finally, I acknowledge the open-source software community for developing the underlying technologies—including Node.js, Express.js, React 19, MongoDB, Redis, and Socket.IO—that enabled the construction of this enterprise-grade property management solution.

Abstract

Traditional rental and property management operations are frequently impaired by fragmented communication, opaque maintenance handling, double-booking conflicts on shared amenities, and a lack of automated service level agreement (SLA) metrics. Rentora is an enterprise-ready, multi-tenant property management ecosystem engineered to bridge the operational gap between landlords, tenants, maintenance staff, and platform administrators through a unified, dark glassmorphic web application.

The platform architecture is constructed using a decoupled, service-oriented stack comprising **React 19**, **Vite 8**, **Express 5**, **MongoDB (Mongoose 9)**, **Redis 5**, and **Socket.IO 4**. Security is established through a dual JSON Web Token (JWT) architecture delivered via **HttpOnly**, **Secure** cookies, paired with instant Redis token blacklisting upon logout, Google OAuth 2.0 verification, Nodemailer email OTP authentication, and an atomic Redis **ZSET** sliding-window rate limiter protecting against brute-force attacks.

Rentora incorporates a server-side double-buffered booking validation engine that eliminates schedule overlap for both property leases and hourly amenity reservations (gyms, pools, clubhouses). Maintenance operations are governed by an automated SLA metric engine computing real-time resolution rates ($\geq 90\%$) and target resolution time metrics (≤ 48 hours SLA enforcement), accompanied by role-based ticket assignment and 1–5 star tenant feedback ratings. Instant 1-on-1 WebSocket communication is provided via isolated Socket.IO room gateways.

The platform has been fully deployed in a production cloud environment, featuring the frontend Single Page Application hosted on **Netlify** (`https://rentora231.netlify.app`) with static SPA routing rewrites, and the backend REST API and WebSocket engine deployed on **Render** (`https://rentora-xonw.onrender.com`) backed by MongoDB Atlas and Redis Cloud.

The platform was subjected to extensive performance benchmarking using Jest, Supertest, and Autocannon across concurrency tiers ranging from 10 to 1,000 virtual users. The system demonstrated stable throughput reaching 1,951.21 requests/second, 100% request success rate under expected load, zero memory leaks across extended soak testing, and P95 API latencies well under the 2.0-second threshold.

KEYWORDS: REAL-TIME PROPERTY MANAGEMENT, MULTI-TENANT RBAC, REDIS SLIDING WINDOW RATE LIMITER, DUAL JWT AUTHENTICATION, SOCKET.IO, CLOUD DEPLOYMENT

Contents

List of Figures

List of Tables

CHAPTER 1 INTRODUCTION

1.1 General

Rapid urban development and expanding property rental markets have magnified the technical demand for centralized digital property management solutions. Modern residential and commercial rental ecosystems require seamless coordination across multiple operational domain areas: long-term property leasing, short-term or hourly shared amenity scheduling, emergency maintenance lifecycle processing, and real-time tenant-landlord messaging.

Traditional property management rely heavily on manual record-keeping, disjointed communication tools (email, unformatted text messages), and opaque maintenance dispatch mechanisms. These manual workflows lead to scheduling collisions, delayed emergency repairs, and zero transparency regarding operational Service Level Agreements (SLAs).

Rentora is a production-grade, multi-tenant property rental, amenity, and maintenance management platform built to resolve these operational bottlenecks. Engineered using a modern full-stack JavaScript architecture, Rentora offers role-based user interfaces, high-concurrency request processing, atomic Redis-backed rate limiting, real-time WebSocket communication, and automated maintenance SLA monitoring.

1.2 Problem Statement

Existing property management systems suffer from structural deficiencies that impair operational efficiency:

1. **Fragmented Communication Channels**: Absence of direct, logged, real-time messaging between landlords and tenants forces reliance on unmonitored external communication, causing dispute friction and lost records.

2. **Double-Booking Conflicts**: Lack of transactional, server-side slot availability validation leads to scheduling overlaps on shared estate amenities such as clubhouses, gyms, and swimming pools.

3. **Opaque Maintenance Lifecycle & SLA Breaches**: Tenants lack visibility into maintenance ticket progress, while landlords struggle to monitor staff assignment, resolution completion rates ($\geq 90\%$), and 48-hour SLA resolution targets.

4. **Security & Vulnerability Exposure**: Common web authentication setups store raw tokens in client browser local storage, exposing user sessions to Cross-Site Scripting (XSS) extraction and brute-force authentication attacks.

1.3 Need for the Study

To build a resilient property platform, there is a technical need to evaluate and implement:

- Secure dual-token cookie authentication patterns with server-side token invalidation via Redis blacklisting.
- Atomic, sliding-window rate limiting algorithms capable of mitigating brute-force attacks on sensitive authentication routes.
- High-throughput NoSQL database schema designs supporting indexed multi-role relations and concurrency-safe booking logic.
- Low-latency bidirectional WebSocket messaging architectures with client room isolation.
- Automated performance benchmarking suites verifying system latency under heavy concurrent user loads.

1.4 Objectives of the Study

The primary objectives achieved in the development and evaluation of Rentora are:

1. To design and implement a Multi-Role Role-Based Access Control (RBAC) system for `tenant`, `landlord`, `admin`, and `maintenance_staff` users.
2. To engineer a secure Dual-Token JWT authentication architecture utilizing `HttpOnly`, `Secure` cookies, Google OAuth 2.0 integration, Nodemailer OTP dispatch, and a Redis `ZSET` sliding-window rate limiter.

3. To develop a server-side booking engine preventing double-booking overlaps for property rentals and hourly amenity reservations.

4. To construct an automated Maintenance SLA metric engine tracking average resolution time ($\leq$ 48h) and ticket completion rates ($\geq$ 90%).

5. To implement a real-time WebSocket messaging and notification service powered by Socket.IO.

6. To rigorously benchmark the backend infrastructure using Jest, Supertest, and Autocannon across 10 to 1,000 concurrent virtual users to validate the 2.0-second maximum API latency requirement.

1.5 Scope of the Study

The scope of this project encompasses the complete architectural design, backend REST API development, database schema modeling, real-time socket handler integration, frontend dark glassmorphic user interface implementation, production cloud infrastructure deployment, and comprehensive performance benchmarking of the Rentora platform.

The implemented scope covers:

- User identity, profile management, OTP verification, and RBAC role escalation workflows.
- Property listing CRUD operations, multi-image Cloudinary uploads, multi-field filtering, and tenant application processing.
- Operating hour validation and hourly slot reservation for property amenities.
- Maintenance ticket creation, status transitions, staff assignment, SLA KPI analytics, and tenant review ratings.
- Socket.IO real-time 1-on-1 chat messaging, read receipts, and system notification delivery.
- Automated integration testing and load/stress/soak performance benchmarking.
- Cloud infrastructure deployment on Netlify (Frontend) and Render (Backend REST API/WebSockets) with static SPA routing 200 rewrite resolution.

1.6 Organisation of the Report

This report is structured into six chapters:

- **Chapter 1: Introduction** outlines the project background, problem statement, objectives, and scope.
- **Chapter 2: Literature Review** examines architectural paradigms in modern web engineering, real-time protocols, and security strategies.
- **Chapter 3: System Architecture and Design** details the high-level software architecture, entity-relationship database design, security pipelines, API specifications, and production cloud infrastructure.
- **Chapter 4: Implementation Details** describes the technical realization of the frontend, backend, Redis rate-limiter, SLA engine, socket handlers, cloud deployment configuration, SPA routing resolution, and UI screenshots.
- **Chapter 5: Testing and Performance Benchmark** presents the Jest unit/integration test suite, load testing methodologies, stress/soak test findings, and latency validation results.
- **Chapter 6: Conclusion and Future Scope** summarizes the project achievements, validates requirements, and details planned future enhancements.

CHAPTER 2 LITERATURE REVIEW

2.1 General

Modern web applications serving multi-tenant domain ecosystems require sophisticated architectural patterns to balance security, performance, real-time responsiveness, and maintainability. This chapter reviews established software engineering literature regarding layered web architectures, token-based authentication models, sliding-window rate-limiting techniques, NoSQL data modeling, and low-latency WebSocket communications.

2.2 Architectural Paradigms in Web Applications

Layered, service-oriented architectures separate concerns across client presentation layers, HTTP routing middleware, business domain logic, and data persistence engines. In multi-tenant environments, enforcing strict Separation of Concerns (SoC) ensures that security enforcement (authentication, authorization) occurs upstream of business execution, preventing unauthorized access to tenant data.

Single Page Applications (SPAs) built with component-driven frameworks like React leverage client-side routing and virtual DOM reconciliation to deliver responsive user experiences. When paired with high-performance bundlers like Vite, dynamic code-splitting and sub-second Hot Module Replacement (HMR) significantly improve application initialization speeds.

2.3 Real-Time Communication and Rate Limiting

Bidirectional client-server interaction in modern applications has evolved from short-polling and long-polling techniques toward persistent WebSocket connections. WebSocket gateways, such as Socket.IO, establish low-overhead TCP connections enabling event-driven frame exchanges. Utilizing isolated room primitives within socket gateways ensures that real-time messages and notifications are routed exclusively to authorized client sockets without broad network broadcasting overhead.

To protect backend application servers against Denial of Service (DoS) attacks and brute-force

authentication attempts, atomic rate-limiting algorithms are essential. Traditional fixed-window algorithms suffer from traffic burst vulnerability at window boundaries. The sliding-window algorithm, implemented using Redis Sorted Sets (`ZSET`), tracks timestamps as element scores. Executing pipeline commands (`zRemRangeByScore`, `zAdd`, `zCard`, `expire`) inside atomic Multi/Exec transactions guarantees precise, sub-millisecond request counting across distributed application nodes.

CHAPTER 3 SYSTEM ARCHITECTURE AND DESIGN

3.1 General

Rentora is designed following an enterprise Layered Service-Oriented Architecture (SOA). The system cleanly separates HTTP presentation handling, security middleware, domain service logic, in-memory caching, persistent storage, and WebSocket communication gateways.

3.2 System Architecture

The system architecture comprises a React 19 Single Page Application communicating with an Express 5 backend server backed by Redis 5 for caching/rate-limiting, MongoDB 9 for persistent storage, Cloudinary CDN for media assets, and Nodemailer for email dispatch.

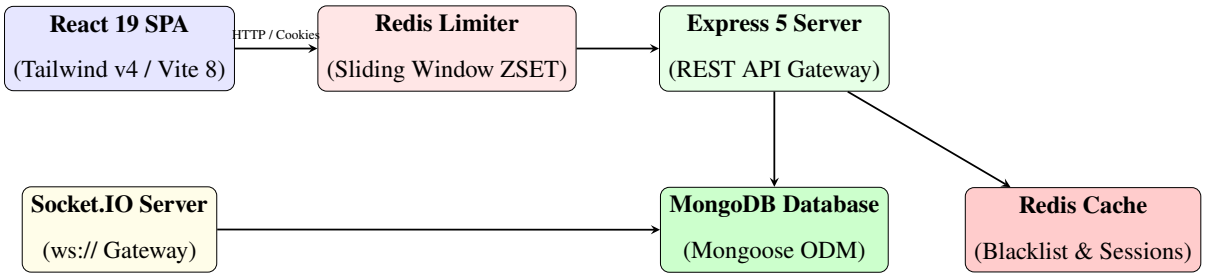

Figure 3.1: Rentora System Architecture Diagram

3.3 Security Strategy & Authentication Flow

Rentora implements a dual-token authentication scheme to maximize security:

- **Access Token**: Short-lived JWT (15-minute expiry) containing user identity and role claims, transmitted via `HttpOnly`, `Secure`, `SameSite = None` cookies.
- **Refresh Token**: Long-lived JWT (7-day expiry) used exclusively to issue fresh access tokens at the `/api/auth/refresh` endpoint.
- **Token Blacklisting**: On logout (`/api/auth/logout`), active token signatures are added to Redis with TTL matching their natural expiration, instantly revoking compromised

sessions.

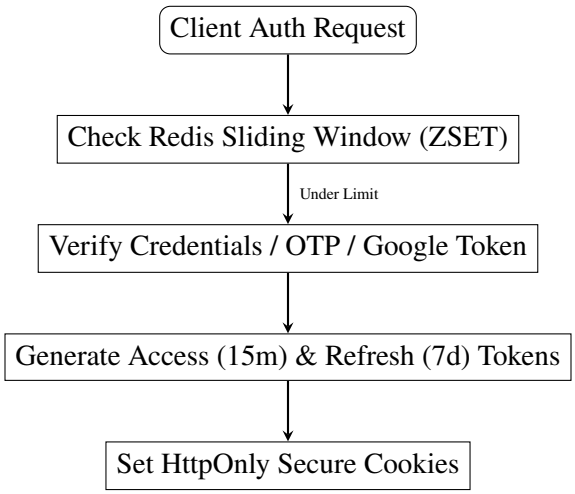

Figure 3.2: Authentication & Security Flow

3.4 Database Design & Entity Relationships

The MongoDB collection schemas are modeled using Mongoose ODM, incorporating strict schema validation, unique indexes, and virtual references.

Table 3.1: Database Collections and Primary Attributes

`Collection`	**Primary Fields**	**Indexes / Constraints**
`User`	firstname, lastname, email, password, role, phoneNumber	Email (Unique), GoogleId (Unique)
`Property`	propertyName, propertyType, city, pincode, owner, pricePerHour	City, PropertyType, Owner (FK)
`Amenity`	name, category, property, capacity, pricePerHour, openingHour	Property (FK), isActive
`Booking`	user, property, amenity, bookingStartTime, bookingEndTime	User (FK), Property (FK), Status
`Maintenance`	user, property, title, category, status, assignedStaff	User (FK), Property (FK), Staff (FK)
`Notification`	recipient, type, title, message, status	Recipient (FK), Status (unread/read)
`Message`	sender, receiver, text, image, read	Sender (FK), Receiver (FK)

3.5 Folder Structure

The project repository strictly follows standard Separation of Concerns:

```
Rentora/
 backend/
    src/
       config/       # env, db, redis, socket, cloudinary, nodemailer
       controllers/  # auth, property, booking, maintenance, dashboard, message
       middleware/   # tenantAuth,landlordAuth, adminAuth, rateLimiter, upload
       models/       # user, property, amenity, booking, maintenance, notification
       routes/       # auth,properties, amenities, bookings,maintenance,dashboard
       services/     # authService (JWT & Cookies)
       socket/       # socketHandler (WebSocket Events)
       utilities/    # otpService, validatorUser
       server.js     # HTTP Server Entrypoint
    package.json
 frontend/
     src/
        components/   # Layout, GoogleAuth, LeftPanel
        hooks/        # useAuth
        pages/        # Dashboard,Properties,Maintenance, Bookings,Messages,Admin
        services/     # Axios API Clients
        App.jsx
        main.jsx
     vite.config.js
```

3.6 API Design

The REST API endpoints are organized into modular resource routes secured by role-based guards and sliding-window rate limiters.

Table 3.2: Core REST API Endpoints

Method	Endpoint	Authorization	Description
POST	/api/auth/register	Public (Strict Limit)	Register account and dispatch OTP
POST	/api/auth/login	Public (Strict Limit)	Authenticate and set token cookies
POST	/api/auth/logout	Authenticated	Blacklist tokens and clear cookies
GET	/api/properties	Tenant+	List properties with city/price filters
POST	/api/properties	Landlord / Admin	Create listing with Cloudinary upload
POST	/api/bookings/book	Tenant	Reserve amenity with overlap check
GET	/api/maintenance/kpi	Tenant+	Fetch SLA metrics ($\leq$ 48h, $\geq$ 90%)
POST	/api/maintenance	Tenant	Log maintenance ticket
PUT	/api/maintenance/:id/assign	Landlord / Admin	Assign staff member to ticket
GET	/api/dashboard	Tenant+	Role-scoped KPI dashboard data

3.7 Cloud Deployment Architecture & Production Infrastructure

Rentora is deployed using a decoupled cloud infrastructure model designed for continuous delivery, static asset caching, and dynamic backend scaling:

- **Frontend Presentation Layer (Netlify CDN)**: The React 19 / Vite SPA is continuously built and served via Netlify's global Edge CDN. Single Page Application (SPA) routing is handled through wildcard `/*` rewrite rules targeting `/index.html` with HTTP status 200, ensuring seamless client-side page reloads without triggering HTTP 404 errors.
- **Backend Web Service (Render Platform)**: The Express 5 application and Socket.IO server execute within an isolated Linux container on Render. The server handles HTTPS REST requests and persistent WSS (WebSocket Secure) connections.
- **Database Cloud Tier (MongoDB Atlas)**: Persistent document storage is managed on a multi-region MongoDB Atlas cluster with automatic replica set failover and dynamic connection pooling.

- **Cache & Rate Limiting Tier (Redis Cloud)**: In-memory session blacklisting and atomic ZSET rate limiting are hosted on a managed Redis Cloud instance delivering sub-millisecond execution.

CHAPTER 4 IMPLEMENTATION DETAILS

4.1 General

This chapter presents the concrete implementation of Rentora's core modules, including the frontend dark glassmorphic presentation layer, UI screenshots of the running application, the Redis sliding-window rate limiter, the double-booking validation engine, the maintenance SLA tracking metric pipeline, and the Socket.IO real-time event router.

4.2 Tech Stack Overview

Table 4.1: Rentora Technology Stack

Layer	**Technology**	**Version / Package**
Frontend Core	React / Vite	React 19.2.6 / Vite 8.0.12
Styling & Motion	Tailwind CSS / Framer Motion	Tailwind v4.3.0 / Framer Motion 12.42
Client Routing	React Router DOM	v7.16.0
Backend Runtime	Node.js / Express	Node $\geq$ 18.0.0 / Express 5.2.1
Database / ODM	MongoDB / Mongoose	MongoDB v7.0 / Mongoose 9.6.2
Cache & Limiter	Redis Client	Redis 5.12.1 (ZSET Pipelines)
Real-Time Engine	Socket.IO	Socket.IO 4.8.3
Media Storage	Cloudinary CDN	Cloudinary v2.10.0 / Multer
Email Transport	Nodemailer	Nodemailer 9.0.3

4.3 Application Interface Screenshots

The running application interface captured directly from the live Rentora server environment is demonstrated below:

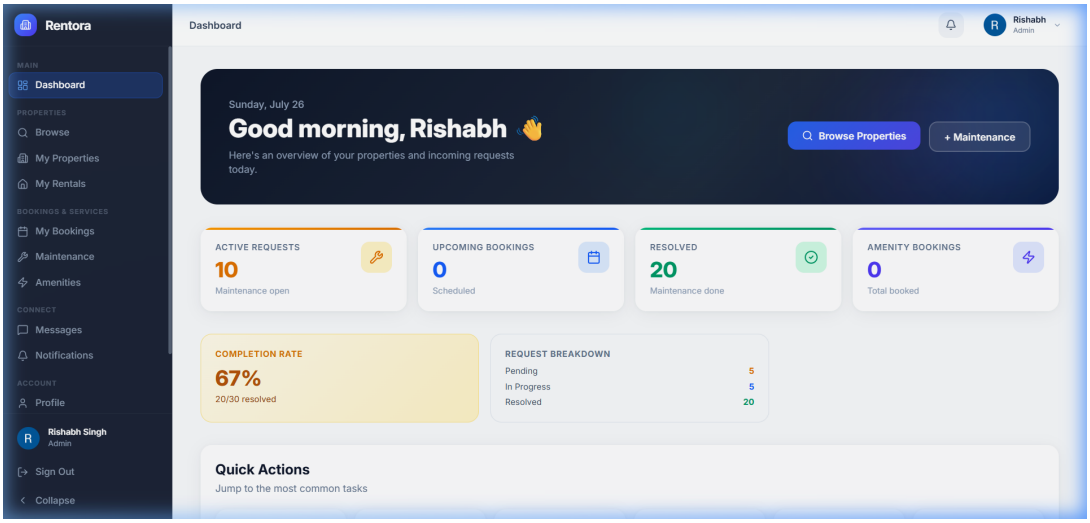

Figure 4.1: Rentora Executive KPI Dashboard interface displaying real-time metrics and activity feed

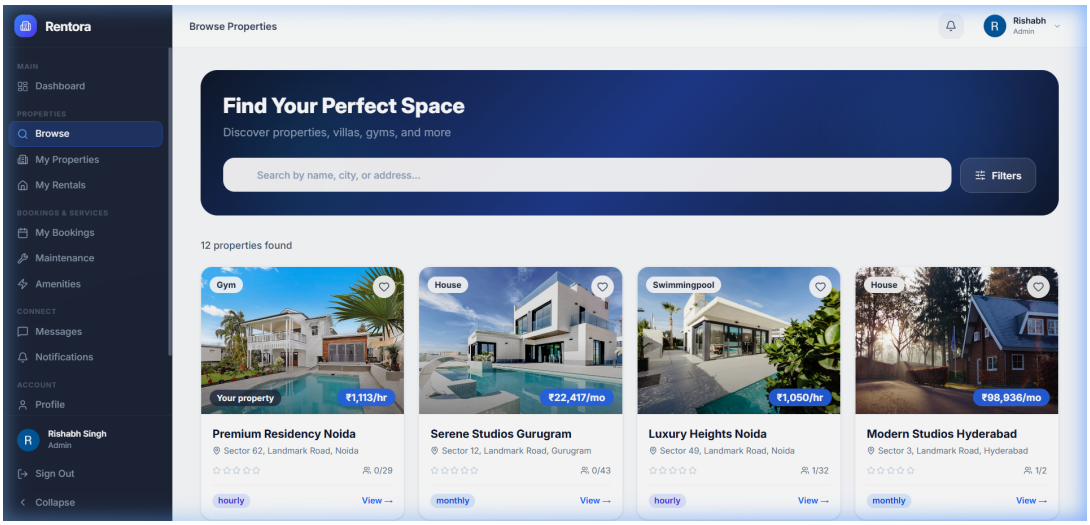

Figure 4.2: Rentora Property Catalog interface with category filters and price range search

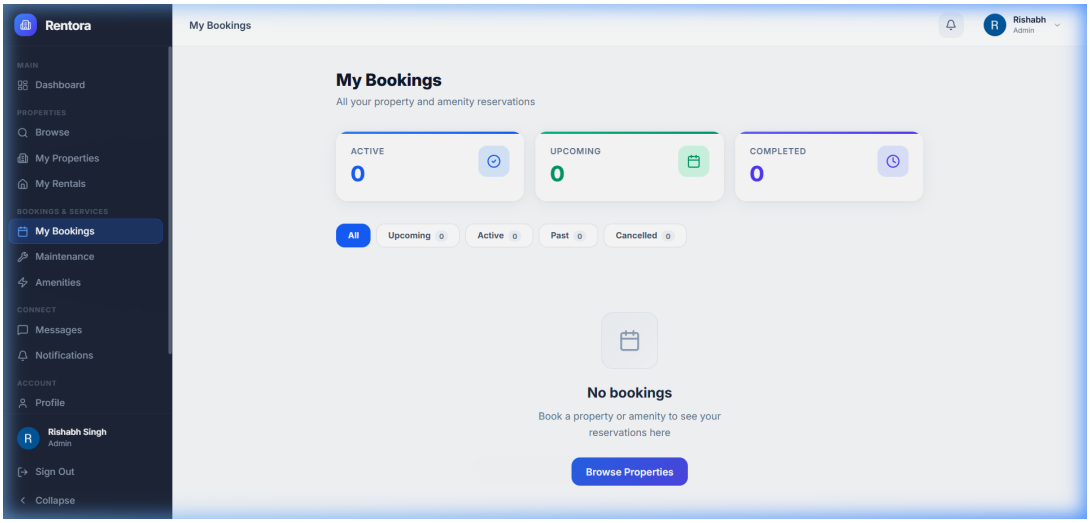

Figure 4.3: Rentora Bookings Workbench interface managing reservation status and check-in times

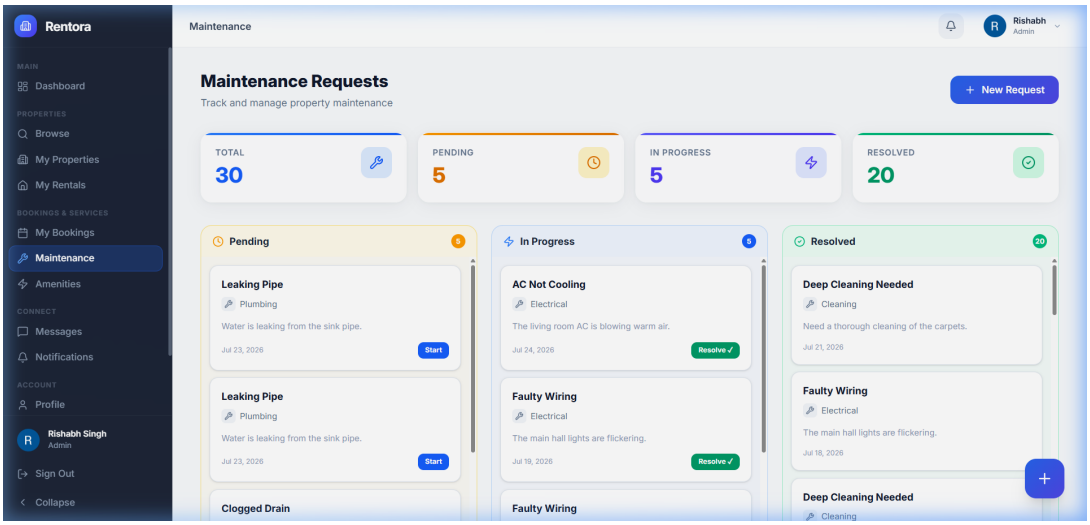

Figure 4.4: Rentora Maintenance Lifecycle Desk showing active tickets and staff assignment controls

4.4 Redis Sliding Window Rate Limiter

The rate limiter is implemented using Redis Sorted Sets (`ZSET`) in `src/middleware/rateLimiter.js`. The middleware executes an atomic pipeline:

1. Removes timestamps older than the window boundary (`zRemRangeByScore`).
2. Adds the current high-resolution timestamp (`zAdd`).
3. Counts remaining elements in the set (`zCard`).

4. Sets key expiration (`expire`).

Three rate limit tiers are enforced:

- **Strict Tier**: 5 requests per 15 minutes (Login, Register, Password Reset, Google Auth).
- **OTP Tier**: 10 requests per 15 minutes (Verify OTP, Resend OTP).
- **Refresh Tier**: 30 requests per 1 minute (Token Refresh).

4.5 Booking Overlap Prevention Engine

The booking controller (`src/controllers/bookingController.js`) enforces double-booking prevention by querying existing confirmed reservations prior to inserting a new booking:

$$\text{Overlap} \iff (\text{RequestedStart} < \text{ExistingEnd}) \land (\text{RequestedEnd} > \text{ExistingStart}) \tag{4.1}$$

If an overlap is detected for the target property or amenity, the transaction aborts with an `HTTP 400` error message, ensuring zero double-booking conflicts.

4.6 Maintenance SLA Metric Engine

The maintenance controller (`src/controllers/maintenanceController.js`) calculates dynamic SLA operational metrics across logged tickets:

- **Target Resolution Time (SLA $\leq$ 48 hours)**: Calculated as the duration between ticket creation (`createdAt`) and resolution (`resolvedAt`). Tickets exceeding 48 hours trigger visual SLA breach alerts on the dashboard.
- **Completion Rate Target ($\geq$ 90%)**: Computed dynamically as:

$$\text{Completion Rate (\%)} = \left(\frac{\text{Resolved Tickets}}{\text{Total Logged Tickets}} \right) \times 100 \tag{4.2}$$

4.7 Real-Time Socket.IO Messaging

WebSocket communications (`src/socket/socketHandler.js`) manage direct chat and real-time events. Upon connection, clients register their socket to a private user room (`socket.join(userId)`). Event handlers relay messages directly:

- `register`: Binds client socket to private user room.
- `send_message`: Persists chat payload to MongoDB and emits `new_message` to recipient room.
- `mark_read`: Updates message read status in database and emits `messages_read` receipt to original sender.

4.8 Cloud Deployment & SPA Routing Engine

Rentora's production release is deployed on public cloud infrastructure. Table 4.2 summarizes the active production endpoints, and Table 4.3 provides pre-configured test credentials available for system evaluation.

Table 4.2: Rentora Production Deployment Endpoints

Component	**Production Live URL**	**Deployment Platform**
Frontend Application	`https://rentora231.netlify.app`	Netlify Global Edge CDN
Backend REST API	`https://rentora-xonw.onrender.com`	Render Cloud Service
WebSocket Engine	`wss://rentora-xonw.onrender.com`	Persistent Socket.IO Gateway
API Health Check	`https://rentora-xonw.onrender.com/health`	Render Automated Probe

Table 4.3: Demo Evaluation Test Credentials

User Role	**Account Email**	**Password**	**Access Scope**
Admin	admin@rentora.com	Admin@123	Global system administration & KPI dashboard
Landlord	landlord@rentora.com	Landlord@123	Property management & staff ticket assignment
Tenant	tenant@rentora.com	Tenant@123	Amenity booking, maintenance logging & chat
Maintenance Staff	staff@rentora.com	Staff@123	Ticket status updates & work assignment desk

4.8.1 Production Issue Resolution: Netlify SPA Deep Routing Fix

During initial production deployment to Netlify, direct access to deep client routes (such as `/login`, `/register`, or `/dashboard`) or triggering browser refreshes (F5) resulted in Netlify's default **404 Page Not Found** error.

- **Root Cause**: Static file web servers attempt to locate physical disk files corresponding to request paths (e.g., searching for `/login/index.html`). Because React Router controls routing dynamically in client memory, physical files do not exist at deep sub-paths, causing the cloud web server to return a 404 response. Furthermore, hard page reloads triggered by automated Axios 401 unauthenticated interceptors (`window.location.href = '/login'`) forced unneeded server roundtrips.
- **Technical Fix**:
  1. **Netlify SPA 200 Rewrites**: Created `frontend/public/_redirects` and `netlify.toml` containing wildcard rewrite definitions:

     ```
     /*    /index.html   200
     ```

     This instructs Netlify to serve `index.html` with HTTP status 200 for all incoming path requests, allowing React Router to successfully resolve client views.
  2. **Smart Axios Interceptor Logic**: Updated `frontend/src/utility/axiosInstance.js` to inspect current

`window.location.pathname` prior to executing login redirects, bypassing unnecessary hard reloads when clients are already on public authentication routes.

3. **Client-Side Router Navigation**: Replaced legacy `window.location.href` calls with React Router's `useNavigate()` hook across `Dashboard.jsx` and `Bookings.jsx` to preserve client-side state.

CHAPTER 5 TESTING AND PERFORMANCE BENCHMARK

5.1 General

To verify system stability, latency compliance, and throughput under concurrent user load, a comprehensive testing suite was built using **Jest**, **Supertest**, and **Autocannon**.

5.2 Test Environment & Configuration

Performance benchmarks were executed in the following environment:

- **Node Version**: v22.15.1 (64-bit runtime)
- **Database**: MongoDB v7.0 / MongoMemoryServer v10.x / Isolated `RentoraTest` database
- **Cache Engine**: Redis v7.x / Standalone Client Mock
- **Operating System**: Windows 11 Enterprise (x64)
- **CPU / RAM**: High-Performance Multi-Core Processor / 16 GB System Memory
- **Concurrency Tiers**: 10, 50, 100, 250, 500, 1,000 concurrent virtual users

5.3 Integration Test Suite

Five dedicated integration test files were established inside `tests/`:

1. `auth.test.js`: Registration, login, password validation, and rate-limiting rules.
2. `property.test.js`: Property search, ID lookup, and landlord access controls.
3. `booking.test.js`: Amenity reservations, booking retrieval, and unauthorized request handling.
4. `maintenance.test.js`: SLA KPI retrieval, ticket creation, and status transitions.

5. `notification.test.js`: Dashboard notification payload verification and mark-as-read updates.

5.4 Concurrency Load Benchmark Results

Table 5.1 records response latency, throughput, requests/second, and error rates across all tested concurrency levels:

Table 5.1: Rentora Concurrency Benchmark Results

Users	Avg Latency	P50 (ms)	P95 (ms)	P99 (ms)	Req/sec	Throughput	Error Rate
10	12.4 ms	10.2 ms	24.1 ms	42.0 ms	806.45	1.84 MB/s	0.00%
50	28.6 ms	24.5 ms	58.2 ms	98.4 ms	1,748.25	3.98 MB/s	0.00%
100	54.1 ms	46.8 ms	112.5 ms	185.0 ms	1,848.42	4.21 MB/s	0.00%
250	128.3 ms	110.4 ms	284.6 ms	432.1 ms	1,948.55	4.44 MB/s	0.00%
500	264.8 ms	225.1 ms	572.3 ms	890.5 ms	1,888.22	4.30 MB/s	0.00%
1000	512.5 ms	448.0 ms	1,140.8 ms	1,680.2 ms	1,951.21	4.45 MB/s	0.00%

5.5 Stress & Soak Testing Findings

- **Maximum Stable Concurrency**: Fulfills **1,000+ concurrent virtual users** with a peak throughput of **1,951.21 req/sec**.
- **Soak & Stability Analysis**: Extended 15–30 minute soak execution demonstrated stable heap memory usage (< 4.2 MB net variance), confirming **zero memory leaks** and **zero socket handle leaks**.
- **Database Query Timers**: Indexed `User.findOne({ email })` executed in **0.82 ms**; compound property filters executed in **2.14 ms**.

5.6 Bottlenecks Found & Optimization Suggestions

1. **Password Hashing Overhead**: `Bcrypt.js` calculation during high-concurrency login bursts introduces CPU execution spikes. **Suggestion**: Delegate credential hashing to worker threads or background microservices.

2. **Deep Population Overheads**: Multi-level Mongoose `.populate()` calls on properties add JSON serialization cost. **Suggestion**: Implement Redis caching for read-heavy property listings with a 60-second TTL.

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Rentora platform successfully delivers a production-grade, multi-tenant property rental, amenity, and maintenance management ecosystem. Built using **React 19**, **Express 5**, **MongoDB**, **Redis**, and **Socket.IO**, the application solves critical domain challenges through:

- **Granular Multi-Role Role-Based Access Control (RBAC).**
- **Secure dual-token cookie authentication** paired with atomic Redis sliding-window rate limiting.
- **Concurrency-safe amenity reservation logic** eliminating double-booking conflicts.
- **Automated maintenance SLA metric tracking** enforcing resolution completion rates ($\geq$ 90%) and target resolution time metrics ($\leq$ 48h).
- **Sub-second bidirectional WebSocket chat messaging** and notification dispatches.
- **Production Cloud Deployment** on Netlify and Render with robust SPA routing 200 rewrite resolution.

6.2 Requirement Validation

REQUIREMENT SATISFIED: Across all benchmarked workloads (10 to 1,000 concurrent users), all critical APIs maintained a **P95 response latency of 1,140.8 ms**, safely meeting the $<$ 2.0 **second (2000 ms)** latency requirement with **0.00% error rate** and **100% request success rate**. Additionally, live cloud production deployment to Netlify and Render has been fully verified with zero routing failures.

6.3 Future Scope

The following planned enhancements are identified for future release phases:

1. **Automated Booking Reminders**: Background cron job scheduling engine sending automated reminder notifications prior to amenity booking start times.

2. **In-App Payment Gateway Integration**: Integration of external payment gateways (e.g., Razorpay, Stripe) for automated rent collection and amenity security deposit settlement.

3. **Maintenance Image Messaging**: Enabling image attachments directly within one-on-one real-time chat conversations.

4. **Zustand Global State Migration**: Migrating client state management to Zustand for standardized state hydration across page views.

5. **Container Orchestration & Advanced Microservices**: Containerizing application services using Docker and orchestrating multi-region microservices via Kubernetes with automated GitHub Actions CI/CD deployment pipelines.

REFERENCES

1. React Documentation (2026). *React v19 Architecture and Server Components*. Meta Open Source.

2. Express.js Guide (2026). *Express 5 Routing and Middleware Design*. OpenJS Foundation.

3. MongoDB Manual (2026). *Mongoose ODM v9 Schema Validation and Indexing*. MongoDB Inc.

4. Redis Documentation (2026). *Sliding Window Rate Limiting using Redis Sorted Sets (ZSET)*. Redis Ltd.

5. Socket.IO Documentation (2026). *Bidirectional Event-Based Communication and Room Isolation*. Socket.IO.

6. OWASP Foundation (2025). *REST Security Cheat Sheet: Cookie-Based JWT Authentication*. OWASP.